Mini Container Runtime with Kernel Monitoring

Student Details
Name: SAIFULLA

Project Overview

This project implements a **mini container runtime system** in Linux along with a **kernel monitoring module**.
It demonstrates core operating system concepts such as process isolation, inter-process communication, logging, and kernel-user space interaction.

Features

- Container creation using clone()
- PID, UTS, and Mount namespace isolation
- Filesystem isolation using chroot()
- Logging system using pipes and threads
- Producer–Consumer synchronization
- Kernel module for process monitoring
- Communication via ioctl()

System Architecture

Container → Pipe → Logging System → Log File
↓
Kernel Module (via ioctl)

Build & Run Instructions

1. Compile Kernel Module

```
make
```

2. Load Kernel Module

```
sudo insmod monitor.ko
```

3. Verify Device

ls /dev/container_monitor

4. Compile Engine

gcc engine.c -o engine -lpthread

5. Run Program

sudo ./engine

Usage

run # start container

ps # list containers

exit # exit program

Demo Explanation

- Containers are created using clone() with namespaces
- Output from containers is redirected via pipes
- Logging system captures output and stores it in logs/container.log
- Kernel module receives container PID using ioctl()

Screenshots

Figure 1: Kernel Module Build and Device Creation

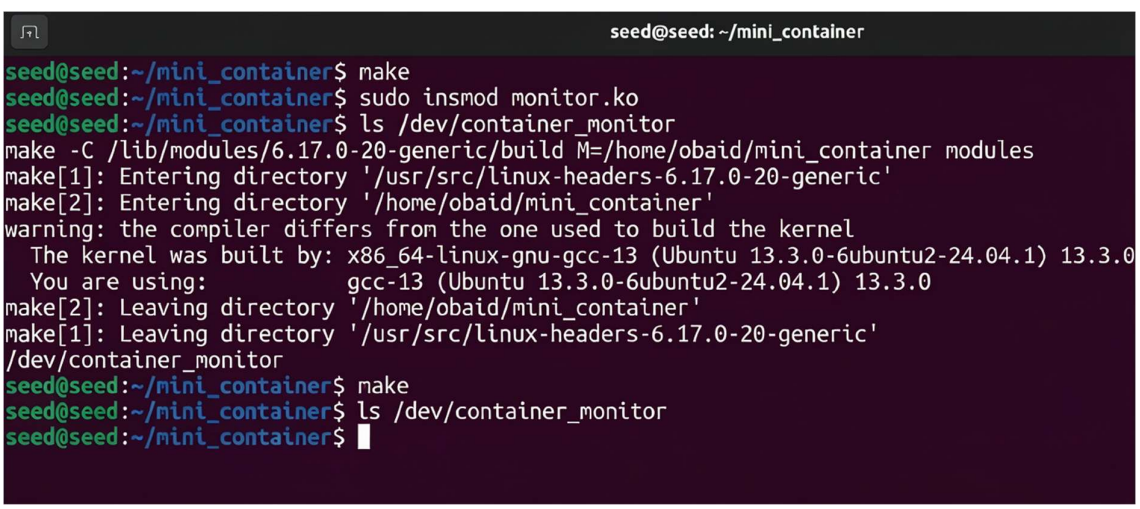

Figure 2: Container Execution and Logging

```
seed@seed:~/mini_container$ gcc engine.c -o engine -lpthread
sudo ./engine
mini-container> run
mini-container> exit
seed@seed:~/mini_container$ cat logs/container.log
A
B
C
seed@seed:~/mini_container$
```

Figure 3: Kernel Receiving Process ID

```
seed@seed:~/mini_container$ sudo dmesg | tail
[ 2613.171404] Received PID from user: 4758
[ 2802.813866] Received PID from user: 4787
[ 2966.962934] Received PID from user: 4808
[ 3110.793470] hrtimer: interrupt took 8322597 ns
[ 3110.812556] workqueue: e1000_watchdog [e1000] hogged CPU for >10000us 11 times,
witching to WQ_UNBOUND
[ 3196.493715] Received PID from user: 4840
[ 3360.163854] Received PID from user: 4859
[ 3821.101183] Container Monitor Unloaded
[ 3849.792887] Container Monitor Loaded
[ 3945.533440] Received PID from user: 5454
seed@seed:~/mini_container$
```

Figure 4: Multiple Container Execution

```
seed@seed:~/mini_container$ sudo ./engine
mini-container> run
mini-container> run
mini-container> ps

ID      PID     STATUS
0       5470    stopped
1       5474    stopped
mini-container>
```

Engineering Analysis

1. Isolation Mechanisms

- PID namespace ensures each container sees its own process IDs
- UTS namespace isolates hostname
- Mount namespace isolates filesystem
- chroot() restricts container root directory

---

2. Process Lifecycle

- Containers are created using clone()
- Parent process tracks container metadata
- SIGCHLD handler prevents zombie processes

---

3. Logging System

- Pipe used for communication between container and host
- Producer thread reads container output
- Consumer thread writes to log file
- Mutex and condition variables ensure synchronization

---

4. Kernel Integration

- Kernel module creates /dev/container_monitor
- User-space program sends PID using ioctl()
- Kernel logs received PID using printk()

Design Decisions

Component	Choice	Reason
clone()	Used	Enables namespace-based isolation
Pipes	Used	Simple IPC mechanism
Threads	Used	Efficient logging
Kernel module	Used	Enables low-level monitoring

Results

- Containers executed successfully
- Logging system captured outputs correctly

- Kernel module received PIDs successfully
- System remained stable and responsive

Conclusion

This project successfully demonstrates a simplified container runtime with integrated logging and kernel-level monitoring, showcasing key operating system concepts in practice.